Isogeny-based Cryptography

Gemma Powell

Contents

1 Introduction **2**

2 Isogenies between elliptic curves **2**
2.1 Examples of isogenies 3
2.2 Constructing isogenies 4

3 Dual isogenies **4**

4 The j-invariant **5**

5 Isogeny graphs of elliptic curves **5**

6 Supersingular Isogeny Diffie-Hellman **6**
6.1 The m-torsion group and the choice of p 8
6.2 Finding generators of an m-torsion subgroup 9
6.3 SIDH example 11

7 Breaking SIDH **12**

A SageMath code for isogeny examples **14**
A.1 Supersingular j-invariant graphs 14
A.2 SIDH graph representation 15
A.3 SIDH example 17

1 Introduction

While many protocols in elliptic curve cryptography (ECC) rely on the hardness of the elliptic curve discrete logarithm problem (ECDLP), elliptic curves admit additional mathematical structures that can be used in cryptography. In particular, isogeny-based cryptography exploits the difficulty of computing certain structure-preserving maps between elliptic curves, known as isogenies. This approach provides a fundamentally different foundation for security.

2 Isogenies between elliptic curves

Let K be a field. A rational function over K is a quotient of polynomials in $K[x]$. If $f(X, Y)$ is a rational function in two variables, it may be viewed as a function on an elliptic curve E by evaluating it at points on the curve [7, p. 338].

Definition 2.1 (Isogeny)**.** [8, p. 386] Let E_1 and E_2 be elliptic curves over a field K. An isogeny $\phi\colon E_1 \to E_2$ is a non-constant rational map defined over K that is also a group homomorphism. Equivalently, for all $P, Q \in E_1(\overline{K})$,

$$\phi(P + Q) = \phi(P) + \phi(Q), \quad \text{and} \quad \phi(\mathcal{O}_{E_1}) = \mathcal{O}_{E_2},$$

where $\overline{K}$ denotes the algebraic closure of K. If such a map exists, we say that E_1 and E_2 are isogenous. If there exists an isogeny $\psi\colon E_2 \to E_1$ such that

$$\psi \circ \phi = \mathrm{Id}_{E_1}, \quad \text{and} \quad \phi \circ \psi = \mathrm{Id}_{E_2},$$

then we say ϕ and ψ are isomorphisms of elliptic curves, and E_1, E_2 are isomorphic.

Intuitively, an isogeny is a structure-preserving map that respects the curve's group law. For elliptic curves in short Weierstrass form, an isogeny $\phi\colon E_1 \to E_2$ can be expressed as

$$\phi(x, y) = (r_1(x), r_2(x)y) = \left(\frac{p(x)}{q(x)}, \frac{s(x)}{t(x)}y\right),$$

for affine points (x, y), where $p, q, s, t \in K[x]$, with p, q coprime and s, t coprime [8, p. 387]. We call this the standard form of ϕ. We can classify these isogenies further.

Definition 2.2 (Degree and separability)**.** [8, p. 387] Let $\phi(x, y) = (r_1(x), r_2(x)y)$ be an isogeny in standard form. Let $r_1(x) = p(x)/q(x)$. The degree of ϕ is

$$\deg(\phi) = \max\{\deg(p), \deg(q)\}.$$

We say that ϕ is separable if $r_1'(x) \not\equiv 0$, otherwise ϕ is inseparable.

The kernel of an isogeny $\phi\colon E_1 \to E_2$ can be defined as

$$\ker(\phi) = \{P \in E_1(\overline{K})\colon \phi(P) = \mathcal{O}_{E_2}\}.$$

Proposition 2.3. [8, p. 387] *Let $\phi\colon E_1 \to E_2$ be an isogeny of elliptic curves E_1 and E_2 defined over a field K. If ϕ is separable, then*

$$\deg(\phi) = \#\ker(\phi).$$

If ϕ is not separable, then

$$\deg(\phi) > \#\ker(\phi).$$

We omit the proof as it involves lengthy calculations, refer to [8] for details.

2.1 Examples of isogenies

A fundamental example that plays a central role in cryptography is multiplication by an integer.

Example 2.4 (Multiplication-by-m)**.** [6, p. 67] Let E be an elliptic curve over a field K. For each $m \in \mathbb{Z}$, we define the multiplication-by-m isogeny

$$[m]\colon E \to E$$

by $[m](P) = \underbrace{P + \cdots + P}_{m \text{ times}}$ if $m > 0$, $[m]P = [-m](-P)$ if $m < 0$, and $[0](P) = \mathcal{O}$, for any $P \in E(K)$.

Proof. Through repeated application of the formulas for point addition, we can express $[m]$ as a rational map on the curve. Suppose $P, Q \in E(\overline{K})$. Then

$$[m](P + Q) = [m](P) + [m](Q)$$

by the group law on elliptic curves. Finally, $[m]\mathcal{O} = \mathcal{O}$ by the definition of the group law. Thus, this is an isogeny. □

For cryptographic applications, separability is essential.

Proposition 2.5. *Let E be an elliptic curve over a field K. Let $m \in \mathbb{Z}$ such that* $\mathrm{char}(K) \nmid m$. *Then, the multiplication-by-m isogeny is separable.*

We omit the proof, as it requires background beyond the scope of this essay; see [6]. In particular, when working over $K = \mathbb{F}_{p^2}$, which has characteristic p, the map $[m]$ is separable when $p \nmid m$.

Example 2.6 (Frobenius map)**.** Let p be a prime. The Frobenius map $\phi_p\colon E \to E$ for an elliptic curve E over $\mathbb{F}_p$ defined as

$$\phi_p(x, y) = (x^p, y^p)$$

is an isogeny.

Proof. Clearly $z \mapsto z^p$ is a rational map, so ϕ_p is also a rational map. Since $\mathbb{F}_p$ has characteristic p, we have $(a + b)^p = a^p + b^p$ for all $a, b \in \mathbb{F}_p$. It follows that for all $P, Q \in E(\overline{\mathbb{F}_p})$, $\phi_p(P + Q) = \phi_p(P) + \phi_p(Q)$. Furthermore, $\phi_p(\mathcal{O}) = \mathcal{O}$ by definition, hence ϕ_p is an isogeny. □

2.2 Constructing isogenies

For cryptography, Vélu's formulas provide a method to construct isogenies with a desired kernel.

Theorem 2.7 (Vélu's Formulas)**.** [8, pp. 392–393] *Let E be an elliptic curve in the form*

$$y^2 + a_1xy + a_3y = x^3 + a_2x^2 + a_4x + a_6,$$

where all a_i are in some field K. This is called generalised Weierstrass form. Let G be a finite subgroup of $E(\overline{K})$. Then there is an elliptic curve E' over K and a separable isogeny $\phi\colon E \to E'$ such that $\ker(\phi) = G$.

Explicit formulas and a full proof are given in [8, pp. 392–393]. For a finite subgroup $G \subset E(\overline{K})$, we define E/G to be the elliptic curve, up to isomorphism, obtained from Vélu's construction.

3 Dual isogenies

An important structural property is that every separable isogeny admits a dual isogeny in the opposite direction.

Theorem 3.1 (Dual isogeny)**.** [6, p. 81] *Let E_1, E_2 be elliptic curves over a field K. Let $\phi\colon E_1 \to E_2$ be a separable isogeny of degree m. Then there is a unique, separable isogeny of degree m,*

$$\hat{\phi}\colon E_2 \to E_1$$

such that $\hat{\phi} \circ \phi = [m]_{E_1}$ and $\phi \circ \hat{\phi} = [m]_{E_2}$. Here $[m]_{E_i}$ is the multiplication-by-m map on the curve E_i for $i = 1, 2$. We call $\hat{\phi}$ the dual isogeny of ϕ.

Proof. We show uniqueness only, for existence see [6, p. 81]. Suppose ϕ' also satisfies the dual properties. Then,

$$(\hat{\phi} - \phi') \circ \phi = (\hat{\phi} \circ \phi) - (\phi' \circ \phi) = [m]_{E_1} - [m]_{E_1} = [0]_{E_1}.$$

Since ϕ is non-constant, it must be surjective [6, p. 66]. This implies $\hat{\phi} - \phi' = 0$. Thus, $\hat{\phi} = \phi'$. □

A special case is the multiplication-by-m isogeny in Example 2.4, which satisfies $\widehat{[m]} = [m]$. Using this, the degree of $[m]$ can be computed [6, p. 83]. Let $d = \deg([m])$ and consider $[d]$. Then,

$$[d] = \widehat{[m]} \circ [m] = [m] \circ [m] = [m^2].$$

Thus, $d = m^2$ as the multiplication-by-m map is uniquely determined by m.

4 The j-invariant

An elliptic curve over a field K is classified by its j-invariant, which depends on the characteristic of K.

Definition 4.1 (j-invariant)**.** [3, p. 142] Let K be a field with $\mathrm{char}(K) \neq 2, 3$. For an elliptic curve E in Weierstrass form over K,

$$E : y^2 = x^3 + ax + b, \quad 4a^3 + 27b^2 \neq 0,$$

the j-invariant of E is

$$j(E) = 1728 \frac{4a^3}{4a^3 + 27b^2}.$$

Analogous formulas exist in characteristics 2 and 3.

The j-invariant of an elliptic curve is unique up to isomorphism, a crucial property for cryptographic applications.

Theorem 4.2. [3, p. 143] *Let K be a field and E_1, E_2 elliptic curves over K. Then, E_1 and E_2 are isomorphic over K if and only if $j(E_1) = j(E_2)$.*

See [6, p. 45] for a proof.

5 Isogeny graphs of elliptic curves

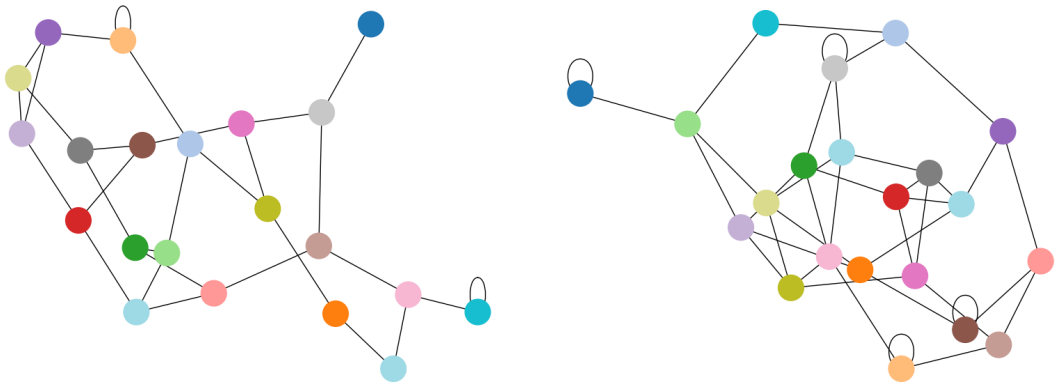
In modern isogeny-based cryptography, protocols consist of walking along paths in specific isogeny graphs.

Definition 5.1 (Isogeny graph)**.** [3, p. 558] An isogeny graph is a graph with vertices being the j-invariants of isogenous curves, and edges being the isogenies between them.

Theorem 3.1 ensures that for every isogeny $\phi\colon E_1 \to E_2$ there is a dual isogeny $\hat{\phi}\colon E_2 \to E_1$ of the same degree. Thus, we typically view isogeny graphs as undirected. For cryptography, we restrict to isogenies of a fixed degree.

Definition 5.2 (ℓ-isogeny graph)**.** [2, p. 9] Let ℓ be a prime number. An ℓ-isogeny graph is an isogeny graph with edges the degree ℓ isogenies only.

Figure 1 shows the 2- and 3-isogeny graphs of specific elliptic curves, known as supersingular, over $\mathbb{F}_{239^2}$. These curves will be defined in the next section and are vital in cryptography. The isogeny graphs shown in Figure 1 were generated using SageMath, the code is provided in Appendix A.1.



(a) The 2-isogeny graph. (b) The 3-isogeny graph.

Figure 1: Supersingular isogeny graphs over $\mathbb{F}_{239^2}$. Different colours represent different j-invariants.

6 Supersingular Isogeny Diffie-Hellman

Throughout this section, we suppose p is a prime number and we consider elliptic curves over $\mathbb{F}_{p^2}$. In cryptography, we restrict to supersingular elliptic curves, which have large and highly connected isogeny graphs, allowing the construction of secure protocols.

Definition 6.1 (m-torsion subgroup)**.** [3, p. 139] Let E be an elliptic curve over a field K. Let m be a positive integer. The m-torsion subgroup is the kernel of $[m]$. That is,

$$E[m] = \{P \in E(\overline{K}) \colon [m]P = \mathcal{O}\}.$$

Definition 6.2 (Supersingular curve)**.** [8, p. 79] Let E be an elliptic curve over a field K such that $\mathrm{char}(K) = p$. We say E is supersingular if the p-torsion subgroup, $E[p]$, is trivial. That is, if $E[p] = \{\mathcal{O}\}$.

Supersingular Isogeny Diffie-Hellman (SIDH), proposed by Jao and De Feo [4], is a key exchange protocol based on random walks in supersingular isogeny graphs. The security of SIDH relies on the difficulty of computing an isogeny between two supersingular elliptic curves given only the curves (or their j-invariants).

Problem 6.3 (Supersingular isogeny path problem)**.** [5, p. 148] *Let p and ℓ be primes. Given E_1 and E_2 supersingular elliptic curves over $\mathbb{F}_{p^2}$, the supersingular isogeny path problem is to find a sequence of isogenies*

$$E_1 \xrightarrow{\phi_1} E_1' \xrightarrow{\phi_2} \cdots \xrightarrow{\phi_n} E_2$$

such that each ϕ_i is a separable isogeny of degree ℓ. Equivalently, determine a path in the ℓ-isogeny graph connecting E_1 to E_2.

Conceptually, the difficulty of this problem arises from the large size of supersingular isogeny graphs. For cryptographic parameters the graph has roughly p vertices, each with only $\ell + 1$ edges. Although individual ℓ-isogenies can be computed efficiently using Vélu's formulas, finding a path between two curves requires searching a vast graph with no known shortcuts, similar to the role of the ECDLP in elliptic curve Diffie-Hellman (ECDH).

We outline the algorithm, as described in [2, pp. 13–14]. Alice and Bob must first agree on various public parameters. They choose a prime

$$p = \ell_A^{e_A} \ell_B^{e_B} - 1,$$

where ℓ_A, ℓ_B are small distinct primes and e_A, e_B are positive integers. In practice, we often take $\ell_A = 2$ and $\ell_B = 3$. Sometimes a small cofactor f is used so that $p = f \cdot \ell_A^{e_A} \ell_B^{e_B} - 1$. The choice of p will be discussed further later.

They must then decide on an initial supersingular curve E over $\mathbb{F}_{p^2}$. Alice's secret isogenies have degree $\ell_A^{e_A}$ and Bob's have degree $\ell_B^{e_B}$. Since a separable isogeny is uniquely determined by its kernel, constructing one of a desired degree requires choosing a subgroup of that order and applying Vélu's formulas.

Public bases are chosen as

$$\langle P_A, Q_A \rangle = E[\ell_A^{e_A}], \quad \langle P_B, Q_B \rangle = E[\ell_B^{e_B}],$$

so that P_A, Q_A have order $\ell_A^{e_A}$ and P_B, Q_B have order $\ell_B^{e_B}$.

Alice selects secret integers $m_A, n_A \in \{0, 1, \ldots, \ell_A^{e_A} - 1\}$ and sets

$$S_A = [m_A]P_A + [n_A]Q_A.$$

She computes a secret isogeny $\phi_A \colon E \to E_A := E/\langle S_A \rangle$, by sequentially composing e_A isogenies of degree ℓ_A. Her public key is

$$\mathrm{PK}_A = (E_A, P'_B, Q'_B) = (\phi_A(E), \phi_A(P_B), \phi_A(Q_B)).$$

Similarly, Bob chooses integers $m_B, n_B \in \{0, 1, \ldots, \ell_B^{e_B} - 1\}$ and sets

$$S_B = [m_B]P_B + [n_B]Q_B.$$

He computes a secret isogeny $\phi_B \colon E \to E_B := E/\langle S_B \rangle$, by composing e_B isogenies of degree ℓ_B. His public key is

$$\mathrm{PK}_B = (E_B, P'_A, Q'_A) = (\phi_B(E), \phi_B(P_A), \phi_B(Q_A)).$$

After exchanging public keys, Alice computes

$$S'_A = [m_A]P'_A + [n_A]Q'_A,$$

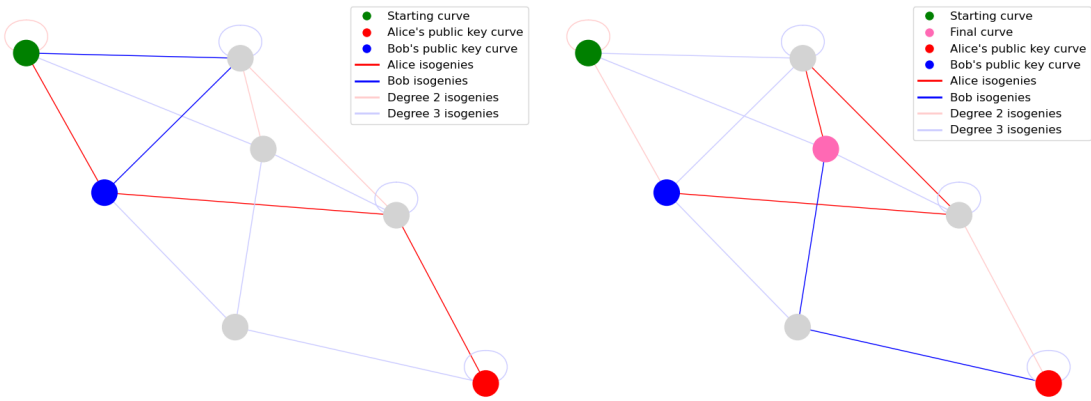
and another secret isogeny $\phi'_A \colon E_B \to E_{AB} := E_B/\langle S'_A \rangle$. She obtains the shared secret as $j_{AB} = j(E_{AB})$, the j-invariant of E_{AB}. Bob similarly computes

$$S'_B = [m_B]P'_B + [n_B]Q'_B,$$

and an isogeny $\phi'_B \colon E_A \to E_{BA} := E_A/\langle S'_B \rangle$, yielding the shared secret as $j_{BA} = j(E_{BA})$.

We can confirm that the curves E_{AB} and E_{BA} are isomorphic, and hence $j_{AB} = j_{BA}$. Through different paths, Alice and Bob construct isogenies whose kernels correspond to the same subgroup of the original curve, and therefore obtain the same j-invariant. The SIDH protocol is summarised in Figure 3.

Remark 6.4 (Graph interpretation)**.** The SIDH protocol can be interpreted as two parties performing random walks in supersingular isogeny graphs. Starting from a common vertex E, Alice takes e_A steps in the ℓ_A-isogeny graph determined by her secret subgroup, while Bob takes e_B steps in the ℓ_B-isogeny graph. Exchanging public keys allows each party to "continue" the other's walk, both arriving at the same final vertex and yielding a shared j-invariant. A small example with $p = 2^3 3^2 - 1 = 71$ is shown in Figure 2, with the corresponding SageMath code in Appendix A.2.



(a) Public key generation. (b) Shared secret generation.

Figure 2: An illustrative example of SIDH over $\mathbb{F}_{71}$, showing Alice and Bob's walks on a supersingular isogeny graph.

Both ECDH and SIDH rely on a difficult mathematical inversion problem, but the underlying structures are different. ECDH operates in the abelian group $E(\mathbb{F}_p)$, while SIDH operates in a non-commutative structure given by paths in an isogeny graph. We now examine how specific steps of the algorithm can be carried out.

6.1 The m-torsion group and the choice of p

Recall, the m-torsion subgroup of an elliptic curve E over a field K is

$$E[m] = \{P \in E(\overline{K}) \colon [m]P = \mathcal{O}\}.$$

From proposition 2.5, if $\mathrm{char}(K) \nmid m$, then $[m]$ is separable. Thus,

$$\#E[m] = \deg([m]) = m^2.$$

Public parameters	Primes ℓ_A, ℓ_B, and a prime $p = \ell_A^{e_A} \cdot \ell_B^{e_B} - 1$, A supersingular elliptic curve E over $\mathbb{F}_{p^2}$ of order $(p-1)^2$, A basis $\langle P_A, Q_A \rangle$ of $E[\ell_A^{e_A}]$, A basis $\langle P_B, Q_B \rangle$ of $E[\ell_B^{e_B}]$,	
	Alice	**Bob**
Pick random secret	$S_A = [m_A]P_A + [n_A]Q_A$	$S_B = [m_B]P_B + [n_B]Q_B$
Compute secret isogeny	$\phi_A \colon E \to E_A = E/\langle S_A \rangle$	$\phi_B \colon E \to E_B = E/\langle S_B \rangle$
Exchange data	$E_A, \phi_A(P_B), \phi_A(Q_B) \longrightarrow$	$\longleftarrow E_B, \phi_B(P_A), \phi_B(Q_A)$
Compute secret subgroup	$S'_A = [m_A]\phi_B(P_A) + [n_A]\phi_B(Q_A)$	$S'_B = [m_B]\phi_A(P_B) + [n_B]\phi_A(Q_B)$
Compute secret isogeny	$\phi'_A \colon E_B \to E_{AB} = E_B/\langle S'_A \rangle$	$\phi'_B \colon E_A \to E_{BA} = E_A/\langle S'_B \rangle$
Compute shared secret	$j_{AB} = j(E_{AB})$	$j_{BA} = j(E_{BA})$

Figure 3: Supersingular Isogeny Diffie-Hellman key exchange protocol

By the fundamental theorem of finite abelian groups, we must have

$$E[m] \cong \mathbb{Z}/m^2\mathbb{Z} \quad \text{or} \quad E[m] \cong \mathbb{Z}/m\mathbb{Z} \times \mathbb{Z}/m\mathbb{Z}.$$

The first case suggests $E[m]$ is cyclic, but every element of $E[m]$ has order dividing m, so cannot generate a cyclic group of order m^2. Thus, we conclude $E[m] \cong \mathbb{Z}/m\mathbb{Z} \times \mathbb{Z}/m\mathbb{Z}$ (see [6, p. 86]). In particular, $E[m]$ is generated by two elements. There exist $P, Q \in E[m]$ such that $E[m] = \langle P, Q \rangle$, and every $R \in E[m]$ can be written uniquely as

$$R = [\alpha]P + [\beta]Q, \quad \alpha, \beta \in \mathbb{Z}/m\mathbb{Z}.$$

Recall, in SIDH we take primes of the form $p = \ell_A^{e_A}\ell_B^{e_B} - 1$. We use supersingular curves satisfying $\#E(\mathbb{F}_{p^2}) = (p+1)^2$, with $E[p+1] \subseteq E(\mathbb{F}_{p^2})$. Since $E[p+1] \cong (\mathbb{Z}/(p+1)\mathbb{Z})^2$, it follows that

$$\#E[p+1] = (p+1)^2 = \#E(\mathbb{F}_{p^2}),$$

and therefore $E[p+1] = E(\mathbb{F}_{p^2})$. Thus, since $p+1 = \ell_A^{e_A}\ell_B^{e_B}$, we have $E[\ell_A^{e_A}], E[\ell_B^{e_B}] \subseteq E(\mathbb{F}_{p^2})$, ensuring the torsion bases for SIDH are defined over $\mathbb{F}_{p^2}$. For more details on the choice of p, see [6].

6.2 Finding generators of an m-torsion subgroup

Over an algebraically closed field, every polynomial factors into linear terms, so a rational function can be written as a ratio of such factors.

Definition 6.5 (Zeros, poles and divisors of rational functions)**.** [7, p. 338] Let f be a rational function in factored form

$$f(X) = \frac{a(X-\alpha_1)^{e_1}(X-\alpha_2)^{e_2}\cdots(X-\alpha_r)^{e_r}}{b(X-\beta_1)^{d_1}(X-\beta_2)^{d_2}\cdots(X-\beta_s)^{d_s}},$$

where the α_i and β_j are distinct. We call the α_i the zeros of f and the β_j the poles, with multiplicities e_i and d_j respectively. The divisor of f is the formal sum

$$\operatorname{div}(f) = e_1[\alpha_1] + \cdots + e_r[\alpha_r] - d_1[\beta_1] - \cdots - d_s[\beta_s].$$

For a rational function on an elliptic curve E, zeros and poles occur at points on E. We can assign multiplicities to get the divisor of f,

$$\operatorname{div}(f) = \sum_{P \in E} n_P[P],$$

where only finitely many integers n_P are non-zero. A central tool for testing linear independence in $E[m]$ is the Weil pairing.

Definition 6.6 (The Weil pairing)**.** [7, p. 340] Let m be an integer, and E an elliptic curve over a field K. For $P, Q \in E[m]$, let f_P and f_Q be rational functions on E such that

$$\operatorname{div}(f_P) = m[P] - m[\mathcal{O}] \quad \text{and} \quad \operatorname{div}(f_Q) = m[Q] - m[\mathcal{O}].$$

Let μ_m denote the mth roots of unity in K^*. The Weil pairing is the map $e_m \colon E[m] \times E[m] \to \mu_m$, given by

$$e_m(P, Q) = \frac{f_P(Q+S)}{f_P(S)} \cdot \frac{f_Q(-S)}{f_Q(P-S)},$$

where S is any point on E such that $S \notin \{\mathcal{O}, P, -Q, P-Q\}$.

It can be shown that e_m is independent of the choices of f_P, f_Q and S, see [6, p. 94]. In SIDH, we need explicit generators of $E[\ell_A^{e_A}]$ and $E[\ell_B^{e_B}]$ to define the secret isogenies. We now define a method to find such generators. Now, suppose $\#E(\mathbb{F}_{p^2}) = (p+1)^2$ and $p \nmid m$. Let

$$h_A = \frac{p+1}{\ell_A^{e_A}}.$$

Choose a random $R \in E(\mathbb{F}_{p^2})$ and set $P = [h_A]R$. If $[\ell_A^{e_A - 1}]P \neq \mathcal{O}$ and $[\ell_A^{e_A}]P = \mathcal{O}$, then P has order $\ell_A^{e_A}$. Repeat with another random point $R' \in E(\mathbb{F}_{p^2})$ to obtain $Q = [h_A]R'$. Instead of attempting to solve $Q = [\alpha]P$, the ECDLP, we test linear independence of P and Q using the Weil pairing.

Theorem 6.7 (Properties of the Weil pairing)**.** [7, p. 341] *Let E be an elliptic curve over a field K. Let m be an integer and e_m the Weil pairing. Then, the following hold.*

(a) The Weil pairing is alternating. That is,

$$e_m(P, P) = 1 \quad \textit{for all } P \in E[m].$$

(b) The Weil pairing is bilinear. That is,

$$\begin{aligned} e_m(P_1 + P_2, Q) &= e_m(P_1, Q)e_m(P_2, Q) \quad \textit{for all } P_1, P_2, Q \in E[m], \\ e_m(P, Q_1 + Q_2) &= e_m(P, Q_1)e_m(P, Q_2) \quad \textit{for all } P, Q_1, Q_2 \in E[m]. \end{aligned}$$

The proof is lengthy, so we omit it and refer the reader to [6, p. 94]. Now, let $e = e_{\ell_A^{e_A}}$. If $Q = [\alpha]P$ for some α, then by the properties in Theorem 6.7,

$$e(P,Q) = e(P, \underbrace{P + \cdots + P}_{\alpha}) = \underbrace{e(P,P) \cdots e(P,P)}_{\alpha} = e(P,P)^{\alpha} = 1.$$

Otherwise, if P and Q are linearly independent, then $e(P,Q) \neq 1$, confirming that $\{P,Q\}$ forms a valid basis of $E[\ell_A^{e_A}]$ and providing the generators needed for SIDH. This process can then be repeated for $E[\ell_B^{e_B}]$. We can now illustrate a complete SIDH key exchange.

6.3 SIDH example

We now give a small illustrative example of the SIDH algorithm implemented in SageMath (full code in Appendix A.3).

Example 6.8 (SIDH)**.** For computational simplicity, calculations are performed over $\mathbb{F}_p$ rather than $\mathbb{F}_{p^2}$. Hence, the final curve coincides with initial curve, though the steps mirror the true SIDH procedure.
Public parameters: Take $(\ell_A, \ell_B) = (2,3), (e_A, e_B) = (6,7)$, giving $p = 139,967$. Let the initial supersingular curve be

$$E : y^2 = x^3 + x.$$

Alice and Bob compute their generators as

$$P_A = [58868 : 6140 : 1], \quad Q_A = [88828 : 16794 : 1],$$
$$P_B = [6226 : 12642 : 1], \quad Q_B = [63166 : 112777 : 1].$$

Alice's public key generation: Choosing $m_A = 16, n_A = 23$, Alice obtains

$$S_A = [m_A]P_A + [n_A]Q_A = [64165 : 57653 : 1],$$

and finds a degree 2^6 isogeny with kernel $\langle S_A \rangle$ to

$$E_A : y^2 = x^3 + 102986x + 83982.$$

Alice's public key is

$$\mathrm{PK}_A = (E_A, \phi_A(P_B), \phi_A(Q_B)) = (E_A, [128644 : 7179 : 1], [68179 : 65102 : 1]).$$

Bob's public key generation: Bob chooses $m_B = 122, n_B = 918$, and computes

$$S_B = [12407 : 65750 : 1],$$

and a degree 3^7 isogeny with kernel $\langle S_B \rangle$ to

$$E_B : y^2 = x^3 + 48519x + 66213.$$

His public key is

$$\mathrm{PK}_B = (\phi_B(E), \phi_B(P_A), \phi_B(Q_A)) = (E_B, [51148 : 136939 : 1], [99401 : 47095 : 1]).$$

Shared secret computation: Alice computes $S'_A = [m_A]\phi_B(P_A) + [n_A]\phi_B(Q_A) = [37301 : 104274 : 1]$ and another degree 2^6 isogeny mapping from E_B to

$$E_{AB} : y^2 = x^3 + x, \quad j(E_{AB}) = 1728.$$

Bob computes $S'_B = [28598 : 20880 : 1]$ and another degree 3^7 isogeny mapping from E_A to

$$E_{BA} : y^2 = x^3 + x, \quad j(E_{BA}) = 1728.$$

Thus, both parties derive the same shared secret.

7 Breaking SIDH

Supersingular Isogeny Diffie–Hellman was long considered a promising post-quantum key exchange protocol due to the assumed hardness of the supersingular isogeny path problem. Its very small key sizes compared to other post-quantum schemes made it appealing for applications with limited storage. However, in 2022, SIDH was broken. In a paper by Castryck and Decru, a new attack showed that the mathematical structure of SIDH leaks enough information to efficiently recover the private key. The problem is reduced to testing candidate isogeny steps in polynomial time [1].

Consequently, the SIDH-based key exchange cryptosystem, a former finalist in the NIST Post-Quantum Cryptography competition, was withdrawn. Despite this, isogeny graphs remain an active research area in post-quantum cryptography. For example, commutative supersingular isogeny Diffie-Hellman (CSIDH) constructs a protocol using a commutative group action on supersingular elliptic curves. Unlike SIDH, it does not require the exchange of torsion point images, and is therefore not vulnerable to the Castryck and Decru attack [1].

References

[1] Wouter Castryck and Thomas Decru. An efficient key recovery attack on SIDH. Cryptology ePrint Archive, Paper 2022/975, 2022. URL `https://eprint.iacr.org/2022/975`.

[2] Craig Costello. Supersingular isogeny key exchange for beginners. Cryptology ePrint Archive, Paper 2019/1321, 2019. URL `https://eprint.iacr.org/2019/1321`.

[3] Steven D. Galbraith. *Mathematics of Public Key Cryptography*. Cambridge University Press, 2012.

[4] David Jao and Luca De Feo. Towards quantum-resistant cryptosystems from supersingular elliptic curve isogenies. In Bo-Yin Yang, editor, *Post-Quantum Cryptography*, pages 19–34. Springer Berlin Heidelberg, 2011.

[5] Kristin E Lauter and Christophe Petit. Supersingular isogeny graphs in cryptography., 2019.

[6] Joseph H Silverman. *The arithmetic of elliptic curves*, volume 106. Springer, 2009.

[7] Joseph H. Silverman, Jill C. Pipher, and Jeffrey Hoffstein. *An introduction to mathematical cryptography*. Springer, second edition, 2014.

[8] Lawrence C. Washington. *Elliptic curves: Number Theory and Cryptography*. Chapman and Hall/CRC, 2 edition, 2008.

A SageMath code for isogeny examples

This appendix contains the code used to generate examples.

A.1 Supersingular j-invariant graphs

The following SageMath code was used to generate the isogeny graphs shown in Example 2.4.

```
#Importing libraries
import networkx as nx
import matplotlib.pyplot as plt
import matplotlib.cm as cm

p = 239 #Prime characteristic
l = 3   #Isogeny degree

#Getting the list of j-invariants
F = GF(p^2)
S = SupersingularModule(p)
j_invariants, _ = S.supersingular_points()

#Finding the edges
edges = []

for j in j_invariants:
    #Setting curve from j-invariant
    E = EllipticCurve_from_j(j)
    #Running through all isogenies of the right degree
    for iso in E.isogenies_prime_degree(l):
        #Adding codomain j-invariants
        j2 = iso.codomain().j_invariant()
        if j2 in j_invariants:
            edges.append((str(j), str(j2)))

#Creating a graph
G = nx.Graph()
G.add_edges_from(edges)

#Sort nodes to fix colours when plotting
nodes = sorted(G.nodes())
cmap = cm.get_cmap('tab20', len(nodes))
color_map = {node: cmap(i) for i, node in enumerate(nodes)}
node_colors = [color_map[n] for n in G.nodes()]

#Drawing the graph
plt.figure(figsize=(8,6))
pos = nx.spring_layout(G)
nx.draw(G, pos,
```

```
        node_color=node_colors,
        edge_color='black',
        node_size=700,
        with_labels=False)
plt.show()
```

A.2 SIDH graph representation

The following SageMath code was used to generate the SIDH graphs in Figure 2.

```
#Importing libraries
import networkx as nx
import matplotlib.pyplot as plt
import matplotlib.cm as cm
from matplotlib.lines import Line2D
import numpy as np
np.random.seed(1)

p = 71 #Prime characteristic
l1 = 2  #Isogeny degrees
l2 = 3

#Getting the list of j-invariants
F = GF(p**2, 'a')
S = SupersingularModule(p)
j_invariants, _ = S.supersingular_points()
print(j_invariants)

j_start = j_invariants[0]
Alice_path = [j_start, F(17), F(40), F(0)]
Bob_path = [j_start, F(48), F(17)]
#Alice_path = [F(17), F(40), F(48), F(66)] Shared secret derivation
#Bob_path = [F(0), F(41), F(66)]

#Finding the edges
edges_l1 = []
for j in j_invariants:
    #Setting curve from j-invariant
    E = EllipticCurve_from_j(j)
    #Running through all isogenies of the right degree
    for iso in E.isogenies_prime_degree(l1):
        #Adding codomain j-invariants
        j2 = iso.codomain().j_invariant()
        if j2 in j_invariants:
            edges_l1.append((str(j), str(j2)))

#Finding the edges
edges_l2 = []
```

```
for j in j_invariants:
    #Setting curve from j-invariant
    E = EllipticCurve_from_j(j)
    #Running through all isogenies of the right degree
    for iso in E.isogenies_prime_degree(l2):
        #Adding codomain j-invariants
        j2 = iso.codomain().j_invariant()
        if j2 in j_invariants:
            edges_l2.append((str(j), str(j2)))

#Creating a graph
G = nx.Graph(loops=True)
G.add_edges_from(edges_l1, degree=2)
G.add_edges_from(edges_l2, degree=3)

#Setting paths
Alice_path = [str(j) for j in Alice_path]
Bob_path = [str(j) for j in Bob_path]
start_node = str(j_start)
#Setting edges
alice_edges = list(zip(Alice_path, Alice_path[1:]))
bob_edges = list(zip(Bob_path, Bob_path[1:]))

#Adding node colours
node_colours = []
for node in G.nodes():
    if node == start_node:
        node_colours.append('green')
    elif node in Alice_path[len(Alice_path)-1]:
        node_colours.append('red')
    elif node in Alice_path[0]:
        node_colours.append('blue')
    elif node in Bob_path[0]:
        node_colours.append('red')
    elif node in Bob_path[len(Bob_path)-1]:
        node_colours.append('blue')
    else:
        node_colours.append('lightgrey')
#Adding edge colours
edge_colours = []
for u, v, data in G.edges(data=True):
    #Alice's walk
    if (u,v) in alice_edges or (v,u) in alice_edges:
        edge_colours.append('red')
    #Bob's walk
    elif (u,v) in bob_edges or (v,u) in bob_edges:
        edge_colours.append('blue')
    #background edges
    elif data["degree"] == 2:
```

```
        edge_colours.append('#ffcccc')
    elif data["degree"] == 3:
        edge_colours.append('#ccccff')

#Adding legend
legend_elements = [
    Line2D([0], [0], marker='o', color='w', markerfacecolor='green',
            markersize=10,  label='Starting curve'),
    #Line2D([0], [0], marker='o', color='w', markerfacecolor='hotpink',
    #        markersize=10, label='Final curve')
    Line2D([0], [0], marker='o', color='w', markerfacecolor='red',
            markersize=10, label="Alice's public key curve"),
    Line2D([0], [0], marker='o', color='w', markerfacecolor='blue',
            markersize=10, label="Bob's public key curve"),
    Line2D([0], [0], color='red', lw=2, label='Alice isogenies'),
    Line2D([0], [0], color='blue', lw=2, label='Bob isogenies'),
    Line2D([0], [0], color='#ffcccc', lw=2, label='Degree 2 isogenies'),
    Line2D([0], [0], color='#ccccff', lw=2, label='Degree 3 isogenies')
]

#Drawing the graph
plt.figure(figsize=(8,6))
pos = nx.spring_layout(G)
nx.draw(G, pos,
        node_color=node_colors,
        edge_color=edge_colors,
        node_size=700,
        with_labels=False)
plt.legend(handles=legend_elements, loc='upper right', fontsize=18)
plt.show()
```

A.3 SIDH example

The following SageMath code was used to generate Example 6.8.

```
#Choosing prime
lA, lB = 2, 3
eA, eB = 6, 7
p = lA^eA * lB^eB - 1

assert p.is_prime()
assert p % 4 == 3 #Needed for below curve to be supersingular

k = GF(p) #We work over GF(p) to keep computations quick
E = EllipticCurve(k, [1, 0])

print("Public parameters:")
print("E0:", E)
```

```
print("E0 is supersingular:", E.is_supersingular()) #Check
print("j invariant of E0:", E.j_invariant())
print(" ")

#Finding curve order
n = E.order()

#Cofactors for subgroup generators
cofactorA = n // lA^eA #Alice
cofactorB = n // lB^eB #Bob'

#Finding Alice's generators:
#PA
while True:
    P = cofactorA * E.random_point() #Pick random point
    if P.order() == lA^eA:
        PA = P
        break
#QA
while True:
    QA = cofactorA * E.random_point()
    #Ensure P and Q independent
    if QA != 0 and (QA * (PA.order() // QA.order()) != PA):
        break

#Finding Bob's generators:
#PB
while True:
    P = cofactorB * E.random_point() #Pick random point
    if P.order() == lB^eB:
        PB = P
        break
#QB
while True:
    QB = cofactorB * E.random_point()
    #Ensure P and Q independent
    if QB != 0 and (QB * (PB.order() // QB.order()) != PB):
        break

print("Subgroup generators:")
print("PA =", PA, ", QA =", QA)
print("PB =", PB, ", QB =", QB)
print(" ")

#Alice public key generation:
mA = randint(0, lA^eA - 1)
nA = randint(0, lA^eA - 1)
SA = mA * PA + nA * QA
phiA = E.isogeny(SA)
```

```
EA = phiA.codomain()

print("Alice's public key generation:")
print("mA =", mA)
print("nA =", nA)
print("SA =", SA)
print("phiA =", phiA)
print("EA =", EA)

#Alice sends to Bob:
EA, phiA_PB, phiA_QB = EA, phiA(PB), phiA(QB)
print("PKA = (", EA, ",", phiA_PB, ",", phiA_QB, ")")
print(" ")

#Bob public key generation:
mB = randint(0, lB^eB - 1)
nB = randint(0, lB^eB - 1)
SB = mB * PB + nB * QB
phiB = E.isogeny(SB)
EB = phiB.codomain()

print("Bob's public key generation:")
print("mB =", mB)
print("nB =", nB)
print("SB =", SB)
print("phiB =", phiB)
print("EB =", EB)

#Bob sends to Alice:
EB, phiB_PA, phiB_QA = EB, phiB(PA), phiB(QA)
EB, phiB_PA, phiB_QA
print("PKB = (", EB, ",", phiB_PA, ",", phiB_QA, ")")
print(" ")

#Alice computes shared secret:
SAB = mA * phiB_PA + nA * phiB_QA
phiAB = EB.isogeny(SAB)
jAB = phiAB.codomain().j_invariant()
print("Alice's shared secret:")
print("SAB =", SAB)
print("phiAB =", phiAB)
print("jAB =", jAB)
print(" ")

#Bob computes shared secret:
SBA = mB * phiA_PB + nB * phiA_QB
phiBA = EA.isogeny(SBA)
jBA = phiBA.codomain().j_invariant()
print("Bob's shared secret:")
```

```
print("SBA =", SBA)
print("phiBA =", phiBA)
print("jBA =", jBA)
print(" ")

#Final check
print("jAB = jBA:", jAB == jBA)
```

The following output confirms that both parties derive the same shared secret.

```
Public parameters:
E0: Elliptic Curve defined by y^2 = x^3 + x over Finite Field of
    size 139967
E0 is supersingular: True
j invariant of E0: 1728

Subgroup generators:
PA = (58868 : 6140 : 1) , QA = (88828 : 16794 : 1)
PB = (6226 : 12642 : 1) , QB = (63166 : 112777 : 1)

Alice's public key generation:
mA = 16
nA = 23
SA = (64165 : 57653 : 1)
phiA = Isogeny of degree 64 from Elliptic Curve defined by
    y^2 = x^3 + x over Finite Field of size 139967 to Elliptic
    Curve defined by y^2 = x^3 + 102986*x + 83982 over Finite Field
    of size 139967
PKA = ( Elliptic Curve defined by y^2 = x^3 + 102986*x + 83982 over
    Finite Field of size 139967 ,
    (128644 : 7179 : 1) , (68179 : 65102 : 1) )

Bob's public key generation:
mB = 122
nB = 918
SB = (12407 : 65750 : 1)
phiB = Isogeny of degree 2187 from Elliptic Curve defined by
    y^2 = x^3 + x over Finite Field of size 139967 to Elliptic
    Curve defined by y^2 = x^3 + 48519*x + 66213 over Finite Field
    of size 139967
PKB = ( Elliptic Curve defined by y^2 = x^3 + 48519*x + 66213 over
    Finite Field of size 139967 ,
    (51148 : 136939 : 1) , (99401 : 47095 : 1) )

Alice's shared secret:
SAB = (37301 : 104274 : 1)
phiAB = Isogeny of degree 64 from Elliptic Curve defined by
     y^2 = x^3 + 48519*x + 66213 over Finite Field of size 139967
    to Elliptic Curve defined by y^2 = x^3 + x over Finite
```

```
    Field of size 139967
jAB = 1728

Bob's shared secret:
SBA = (28598 : 20880 : 1)
phiBA = Isogeny of degree 2187 from Elliptic Curve defined by
    y^2 = x^3 + 102986*x + 83982 over Finite Field of size 139967
    to Elliptic Curve defined by y^2 = x^3 + x over Finite
    Field of size 139967
jBA = 1728

jAB = jBA: True
```